Méthodologies SDLC

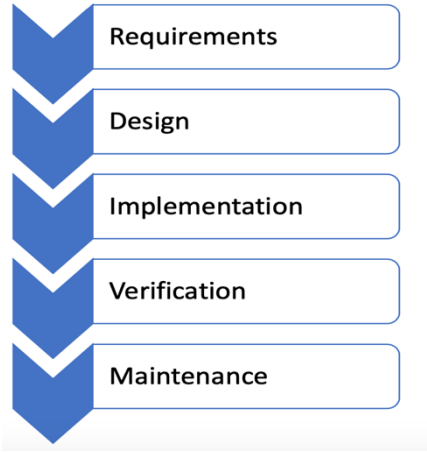
- WaterFall
- Cycle en V
- Itératif et incrémental
- Agile

WaterFall

est désignée comme une méthode de gestion de projet traditionnelle. Parmi les différentes méthodologies existantes, il s'agit de la méthode la plus fréquemment utilisée.

dès qu'une étape du projet est terminée, l'équipe passe à l'étape suivante, il n'y a pas (ou peu) de retour en arrière.

Le modèle en cascade(waterfall) consiste à suivre une succession d'étapes prédéfinies de façon séquentielle, chaque étape menant à la suivante.



Exemple : on peut considérer une voiture . la voiture ne peut pas être peinte si elle n'est pas totalement assemblée, et l'intérieur ne peut pas être monté si elle n'est pas entièrement peinte, etc. Le client ne peut pas intervenir dans le processus de production. Il reçoit la voiture une fois que celle-ci est terminée.

Aventages :

1. **simple, facile à mettre en place, logique et structurée**.

2. **facilite (en théorie) l'estimation précise du budget, du temps et des ressources nécessaires à l'accomplissement de l'ensemble du projet**.

Inconvénients :

1. **manque de flexibilité** à cause de son déroulement séquentiel

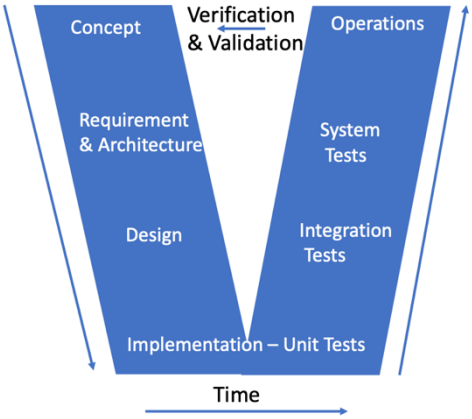
2. **Les risques de déception du client sont plus grands**. Puisque celui-ci ne voit le produit qu'à la livraison

V-cycle :

Évolution du modèle de cascade(waterfsall). la méthode waterfall ne permet pas de retour aux étapes précédentes. Par contre, le cycle en V rend possible le retour en arrière même lorsque l'étape a été validée.

Traditionnellement, le cycle en V découpe le projet en ***3 grosses phases***, à savoir

- **la conception :** analyse des besoins et étude de faisabilité , architecture , désigne
- **la réalisation**
- **la validation** :tests unitaires ,test d'integration,test système , opération



Chaque étape de la droite correspond à
la vérification et la validation de celle de gauche

ce modèle n'est plus autant utilisé qu'avant en entreprise. En effet, sa mise en application reste fastidieuse, la méthode manquant de souplesse par rapport à la gestion de projet Agile

aventage :
il évite les allers-retours durant le **cycle de vie** du projet
il apporte plus de précisions durant sa phase de test.

Agile : as un processus de développement mais **plutôt un état d'esprit :**

- **Individus et interactions sur processus et outils** : Lorsque vous permettez à chaque personne d'apporter une valeur unique à votre projet de développement logiciel, le résultat peut être puissant.
  Si vous appréciez les processus et les outils, les avantages sont que les processus sont généralement clairs et bien compris, et que vous disposez d'un enregistrement écrit des communications sur le projet. Cependant, si vous accordez une grande importance aux individus et aux interactions, vous obtenez de plus grands avantages comme La communication est claire et efficace et les équipes de développement ont plus de chances d'innover

- **Logiciel de travail sur une documentation complète :** Auparavant, les développeurs de logiciels passaient des années à créer une documentation détaillée. C'était avant même de commencer à écrire une seule ligne de code. Et bien que la documentation ne soit pas une mauvaise chose, il arrive un moment où vous devriez vous concentrer sur la fourniture à vos clients de logiciels fonctionnels.

- **Collaboration client sur négociation de contrat** : l'accent doit être mis sur le développement continu. Vous devez créer une boucle de rétroaction avec vos clients afin de vous assurer en permanence que votre produit fonctionne pour eux.

- **Répondre au changement au sujet d'un plan :** nous ne vivons pas dans un monde statique. Les besoins et les exigences changent constamment et les priorités changent constamment. C'est pourquoi une équipe logicielle doit avoir la capacité de pivoter et de changer de direction chaque fois qu'elle en a besoin, avec une feuille de route flexible qui reflète cela. Une feuille de route dynamique peut changer d'un trimestre à l'autre, parfois même d'un mois à l'autre, et les équipes agiles sont capables de suivre ces changements.

1. **Accueillez les besoins changeants, même tard dans le développement**. Les processus agiles exploitent le changement pour l'avantage concurrentiel du client.

2. **Fournissez fréquemment des logiciels fonctionnels,** de quelques semaines à quelques mois, **avec une préférence pour les délais plus courts.**

3. **Les gens d'affaires et les développeurs doivent travailler ensemble quotidiennement** tout au long du projet.

4. **Construisez des projets autour d'individus motivés**. Offrez-leur l'environnement et le soutien dont ils ont besoin et faites-leur confiance pour faire le travail.

5. **La méthode la plus efficace pour transmettre des informations à et au sein d'une équipe de développement est la conversation en face à face**.

6. **Le logiciel de travail est la principale mesure des progrès**.

7. Les processus **agiles favorisent le développement durable**. Les sponsors, les développeurs et les utilisateurs devraient être en mesure de **maintenir un rythme constant indéfiniment.**

8. Une **attention continue à l'excellence technique** et à une bonne conception améliore l'agilité.

9. **La simplicité -** l'art de **maximiser la quantité de travail non effectuée est essentielle**.

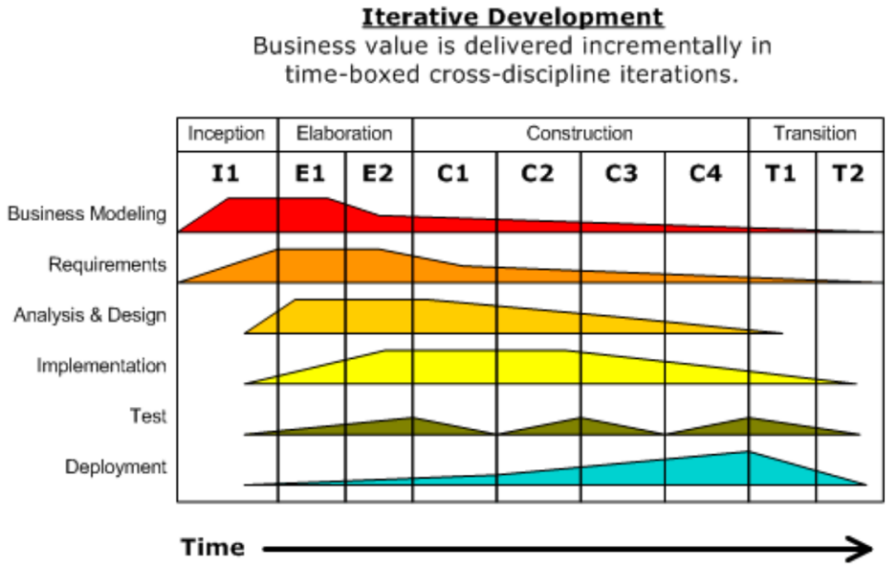
10. **Les meilleures architectures, exigences et conceptions émergent d'équipes auto-organisées**.

11. **À intervalles réguliers, l'équipe réfléchit à la façon de devenir plus efficace**, puis ajuste son comportement en conséquence.

Itératif et incrémental :

Développer un logiciel **par cycles répétés (itératifs)** et **par petits morceaux** (incrémentaux**).**
Le client est impliqué dans chaque incrément .
Rational Unified Process est l'implémentation la plus connue.

RUP consiste à **prioriser d'abord la chose la plus risquée.**
Début: phase plus courte
Élaboration: aborder les facteurs de risque connus
Construction: phase la plus importante, développement
Transition: obtenir les commentaires des utilisateurs


Iterative Development
Business value is delivered incrementally in time-boxed cross-discipline iterations.
Inception
Elaboration
Construction
Transition
I1
E1
E2
C1
C2
C3
C4
T1
T2
Business Modeling
Requirements
Analysis & Design
Implementation
Test
Deployment
Time